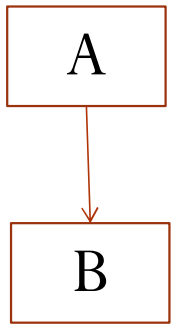
Inheritance

Prepared By Deepali Sonawane, MIT-WPU, Pune

Inheritance

- The mechanism of creating new classes from an existing class is called as **inheritance.**
- Inheritance is the process by which one object can acquire and use the properties of another object.
- The existing class is known as **base class** or **super class** or **parent class**.
- The new class is known as **derived class** or **sub class** or **child class**.
- The derived class shares some of the properties of the base class. Therefore, a code from a base class can be reused by a derived class.

Base class/Super class/ Parent class



Derived class/Sub class / child class

Advantages and Disadvantages of Inheritance

Advantages of Inheritance

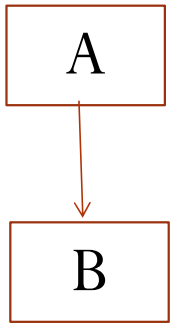
1. **Reusability** - Facility to use public methods of base class without rewriting the same.
2. **Extensibility** - Extending the base class logic as per business logic of the derived class.
3. **Data hiding** - Base class can decide to keep some data private so that it cannot be altered by the derived class
4. **Overriding** -With inheritance, we will be able to override the methods of the base class so that meaningful implementation of the base class method can be designed in the derived class.

Disadvantages of Inheritance

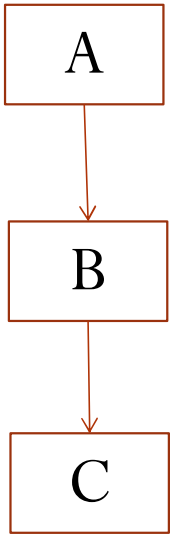
1. Inherited functions work slower than normal function as there is indirection.
2. Improper use of inheritance may lead to wrong solutions.
3. Often, data members in the base class are left unused which may lead to memory wastage.
4. Inheritance increases the coupling between base class and derived class. A change in base class will affect all the child classes.

Types of Inheritance

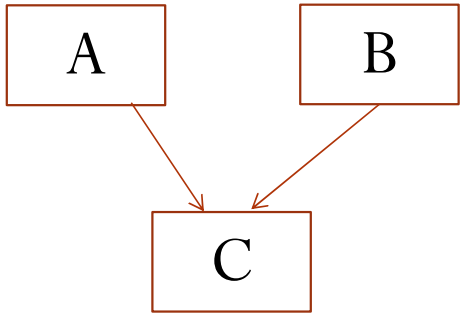
1.Single Inheritance



2.Multilevel Inheritance

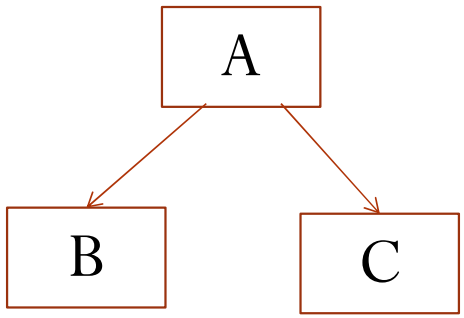


3.Multiple Inheritance

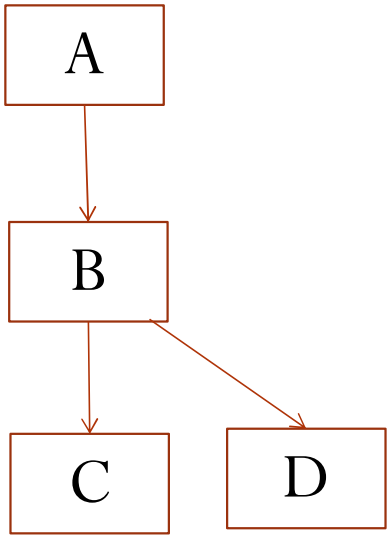


Types of Inheritance

4. Hierarchical Inheritance



5.Hybrid Inheritance



Syntax:

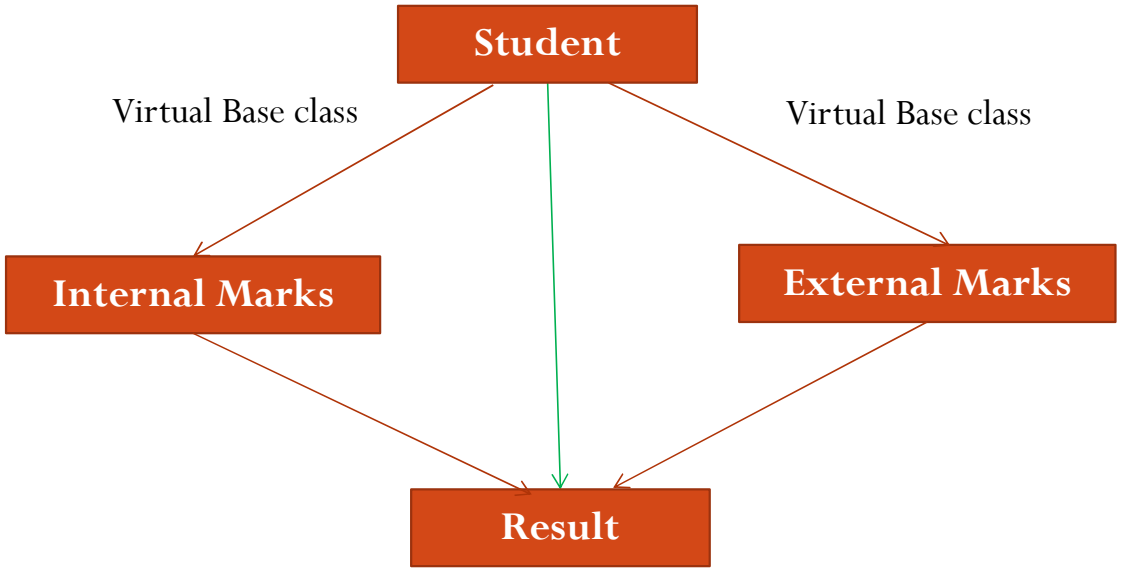
```
class derived_class_name : private/public/protected base_class_name
{
   data members;
}
```

Rules for Derivation Type

Derivation Type	Base Class Members	Access in Derived Class
private	private	Inaccessible
	Public	private
	protected	private
public	private	Inaccessible
	public	public
	protected	protected
protected	private	Inaccessible
	public	protected
	protected	protected

Virtual base classes

- Virtual base classes are useful method **to avoid unnecessary repetition** of the same data member in the multiple inheritance hierarchies.
- To resolve **Diamond problem**, we use **virtual base class concept**.
- Example:



Virtual base classes

```
class Student
{
};
class Internal_marks : public virtual Student
{
};
Class External_marks : virtual public  Student
{
};
class Result : public Internal_marks, public External_marks
{
};
```

Constructors in Derived classes

- We know that the constructor function is used **to initialize the objects**.
- Whenever object of a class is created, a constructor member function is invoked **automatically.**
- If any class contains a constructor with one or more arguments, then it is compulsory for the derived class to have a constructor and pass the arguments to the base class constructors.
- While applying inheritance we usually create objects using the derived class. Thus, it makes sense for the **derived class to pass arguments** to the base class constructor.
- When both derived and base classes contains constructor, the **base constructor** is executed **first** and then the constructor in the **derived class** is executed.

Destructors in Derived classes

- Whenever an object of the class is getting destroyed, the destructors are used to **free the heap area** so that the free memory space may be used subsequently.
- It has been seen that **constructors in hierarchy fire from a base class to a derived class.**
- Destructor in hierarchy fire from **a derived class to a base class order** i.e. they fire in the reverse order of that of the constructors.

Constructor and Destructors in Derived classes

```
class A
{
    int a;
    public:
    A(int a)
    {
        this->a=a;
        cout<<"\nBase class Constructor";
    }
    ~A()
    {
        cout<<"\nValue of a="<<a;
        cout<<"\nBase class Destructor";
    }
};
```

```
class B : public A
{
    int b;
    public:
    B(int a, int b) : A(a)
    {
        this->b =b;
        cout<<"\nDerived class Constructor";
    }
    ~B()
    {
        cout<<"\nValue of b="<<b;
        cout<<"\nDerived class Destructor";
    }
};
int main()
{
    B b(10,20);
    return 0;
}
```

Virtual Functions

- **Virtual functions** are functions that ensure that the **correct function** is called for an object, regardless of the expression used to make the function call.
- When a function is made **virtual**, CPP determines which function to use at run time based on the type of object pointed to by the base pointer, rather than the type of the pointer.
- To make a member function virtual, the keyword **virtual** is used in the methods while it is declared in the class definition but not in the member function definition.
- The keyword virtual should be preceded by a **return type of the function name.**
- The compiler gets information from the keyword virtual that it is a **virtual function** and not a conventional function declaration.

Example of Virtual Functions

```
class Base
{
   public:
   virtual void display()
   {
      cout<<"\nI am in Base class
display";
   }
};
class Derived : public Base
{
   public:
   void display()
   {
      cout<<"\nI am in Derived class
display";
   }
};
```

```
int main()
{
   Base *bp;
   Base b;
   Derived d;
   bp=&b;
   bp->display();
   bp=&d;
   bp->display();
   return 0;
}
```

- Output

I am in Base class display

I am in Derived class display

Pure virtual functions

- The functions which are **only declared** but not defined in the base class are called **pure virtual functions**.
- A function is made pure virtual by preceding its declaration with keyword **virtual** and by post fixing it with **=0**.
- **Syntax: virtual return_type function_name([argument list])=0;**
- A class containing one or more pure virtual functions cannot be used to define an object. The class is therefore only useful as a base class to be inherited into a useable derived class. This class is called as **abstract class**.

Example of Pure virtual functions

```
#include <iostream>
using namespace std;
class Shape
{
public:
   virtual void draw() = 0; // Pure virtual function
};
class Circle : public Shape
{
public:
   void draw() override
   {
      cout << "Drawing Circle\n";
   }
};

int main()
{
Shape* s = new Circle();
s->draw();

 delete s;
}
```

Abstract Class

- A class that contains at least one pure virtual function is considered as an **abstract class.**
- An abstract class is one that **is not used to create objects.**

Example of Abstract Class

```
class shape//abstract class
{
   public:
   virtual void area()=0; //Pure virtual function
};
class sqaure : public shape
{
   int side;
   public:
   sqaure(int side)
   {
      this->side=side;
   }
   void area()
   {
      cout<<"\nArea of sqaure = "<<side*side;
   }
};
```

```
class rectangle : public shape
{
   int l,b;
   public:
   rectangle(int l,int b)
   {
      this->l=l;
      this->b=b;
   }
   void area()
   {
      cout<<"\nArea of rectangle = "<<l*b;
   }
};
int main()
{
   sqaure s(10);
   s.area();
   rectangle r(10,20);
   r.area();
   shape sh;// Error : Can't create object as shape is abstract class
   return 0;
}
```

Thank you